

10. System Architecture

10.1 Architectural Style

The application is a three-tier system. A single-page browser client talks over HTTP to a stateless REST service, which in turn owns a relational database and calls a machine learning module. The tiers are deployed as separate containers and composed with Docker Compose.

Choosing three tiers rather than two was mainly a question of where the model should live. Embedding a model in the browser would have required exporting it to a format JavaScript can execute and would have shipped the model, and therefore its decision boundaries, to every visitor. Keeping it behind the service means the model file never leaves the server, predictions can be logged, and the model can be replaced without a frontend release.

10.2 Presentation Tier

The client is a React application written in TypeScript, bundled by Vite and styled with Tailwind. Recharts draws the radial risk gauge, the SHAP attribution bars and the analytics distributions. Routing is client-side across five views: landing, assessment, results, history and analytics.

The client holds no business logic. Every number it displays arrives from the service, and its only responsibility beyond rendering is input validation, which it performs with Zod schemas whose enumerated values mirror the backend's. This duplication is deliberate: the frontend check exists to give the user immediate feedback, and the backend check exists because a browser cannot be trusted. The backend schema is authoritative.

In production the built assets are served by nginx, which also proxies `/api/` to the service and rewrites unmatched paths to `index.html` so that client-side routes survive a page reload.

10.3 Application Tier

The service is built with FastAPI. Its structure follows a conventional layering, and the layers are enforced by convention rather than by tooling:

- **Routers** (`app/routers/`) accept requests and shape responses. They contain no database access and no computation.
- **Services** (`app/services/`) hold the logic. `assessment_service` owns every database operation; `ml_service` is a thin singleton wrapping the inference module.
- **Schemas** (`app/schemas/`) define request and response contracts as Pydantic models. All validation lives here.
- **Models** (`app/models/`) declare the SQLAlchemy ORM entities.

Seven routes are exposed:

Method	Path	Persists?	Purpose
GET	/health	no	model and database liveness
GET	/model/info	no	training metrics and metadata
POST	/predict	no	prediction without storage
POST	/assessment	yes	prediction and storage
GET	/assessment/{id}	no	retrieve a stored assessment
GET	/assessments	no	paginated list
GET	/analytics	no	aggregate statistics

Two prediction endpoints exist because the two use cases differ. `POST /predict` is stateless and suits exploratory use, where a caller wants a figure without leaving a record. `POST /assessment` performs the same prediction and additionally persists it, returning an identifier the client can later resolve. The assessment form submits to the latter; an earlier build submitted to the former and stored nothing, as Section 19.5 recounts.

The four assessment and prediction endpoints wrap their payload in an envelope of the form `{"success": true, "data": {...}}`. The three remaining endpoints — `/analytics`, `/health` and `/model/info` — return their object directly. The inconsistency is unintended and is discussed in Section 18.3; the client compensates by unwrapping in the API layer on a per-endpoint basis, so no page component is aware of the difference.

A middleware records each request's method, path, status and duration, and attempts to write an audit row. The write is wrapped in a bare exception handler so that a failure to log cannot fail the request.

10.4 Machine Learning Module

The ML module is not a service. It is a Python package that the application tier imports and calls in-process, which avoids a network hop and a serialisation round-trip on every prediction. The trade-off is that the two tiers must share a Python runtime and cannot be scaled independently. For the request volumes this application is designed for, that trade is comfortably favourable.

The module resolves its own location from the `ML_PATH` environment variable, defaulting to `/ml`, which is where Docker Compose mounts the directory. Model artefacts, encoders and the fitted preprocessing constants are loaded once at process start and held in a module-level cache.

10.5 Data Tier

PostgreSQL 15 backs the deployed system. SQLite is supported for local development, and the two are reached through the same SQLAlchemy session factory. One query — the daily assessment count in `/analytics` — cannot be written portably, because SQLite spells date truncation `strftime` and PostgreSQL spells it `to_char`. The service inspects the active dialect and selects the appropriate expression rather than assuming one backend.

Section 17 covers the schema.

10.6 Request Lifecycle

The description below traces `POST /assessment`, the path the assessment form submits to and the one that exercises every component. `POST /predict` follows an identical route up to the point of persistence, then returns without writing.

A submitted assessment moves through the system as follows. The browser serialises the twenty-six form fields into a JSON body and posts it. FastAPI parses the body against `AssessmentInput`; a categorical value outside the permitted literals, or a numeric value outside its declared bounds, terminates the request with a 422 and a field-level error. The validated model is converted to a dictionary and passed to `ml_service.predict`.

Inside the inference module, the input is clipped to the training IQR bounds, the seven derived features are computed using the persisted normalisation constants, the categoricals are encoded using the persisted maps, and the resulting row is reindexed to the exact column order recorded at training time, with any absent one-hot column filled with zero. The Random Forest produces a probability vector across the three classes; the argmax gives the label and its probability gives the confidence. A `TreeExplainer` computes SHAP values for the row. Two rule-based functions inspect the raw inputs and the predicted class to produce a list of behavioural biases and a set of written recommendations.

The service writes an `assessments` row, a `predictions` row and three `behavioral_scores` rows, commits, and returns the assessment identifier with the prediction payload. The middleware appends an audit row. The client stores the payload and navigates to the results view.

Section 11 gives the diagrams for both the deployment structure and this request sequence.

Figure 10.1 — *Request sequence for `POST /assessment`*. Render from the sequence-diagram source in Section 11.2. This is the figure that best demonstrates the system works end to end. Place in Section 10.6.

Screenshot 10.1 — *Swagger UI at `http://localhost:8000/docs`*, with the seven endpoints collapsed so all are visible in one frame. Requires Docker to be running. Place at the end of Section 10.3.